SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study — Answer Key

COURSE NAME: Cloud Computing and Big Data Analytics
COURSE CODE: CSA15
NAME: Pragadeesh Nalan S V
REG NO: 192321161
COURSE OUTCOME COVERED: CO3 — Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)

Answer 1: Smart Retail Inventory Management System

1. Understanding of the Case

- Multiple store devices (POS systems, dashboards) need real-time access to a shared inventory dataset.
- The system must support product data storage, transaction logging, and analytics for demand forecasting.
- Access must be secured across distributed store locations connecting to a centralized cloud back end.

2. Application of Cloud Concepts

- Cloud storage: A managed object/relational storage service (e.g., a cloud SQL database for transactional data plus object storage for receipts/logs) holds product data, transaction logs, and analytics datasets centrally.
- Web APIs: RESTful APIs expose inventory operations (stock lookup, sale updates, restock requests) so POS systems and dashboards can read and write data without direct database access.
- Platform services: A managed analytics/ML service consumes the stored transaction data to run demand forecasting models and generate restocking recommendations.
- Security: An API gateway centralizes authentication and rate limiting; VPNs connect store networks to the cloud back end; firewalls restrict traffic to approved sources.

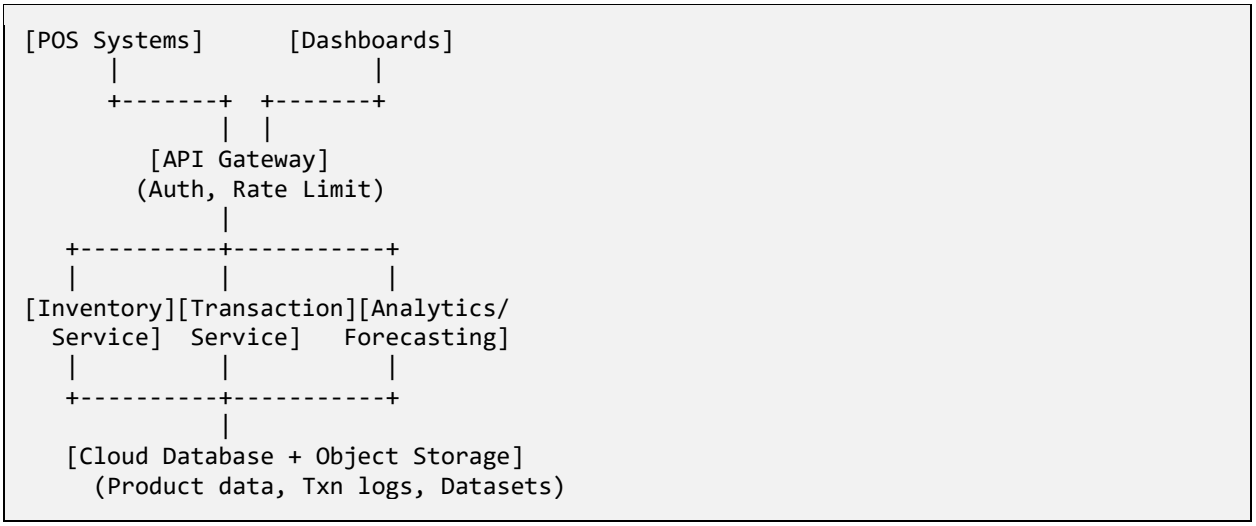
3. Analysis and Trade-offs

- Centralizing inventory in the cloud improves consistency across stores but introduces a dependency on network connectivity — store POS systems need an offline-tolerant local cache with periodic sync to avoid downtime during outages.
- Using a managed platform service for forecasting reduces development effort and infrastructure management compared to building a custom ML pipeline, at the cost of some flexibility and potential vendor lock-in.
- Routing every device through an API gateway adds a small latency overhead but is justified by the security and monitoring benefits (centralized logging, throttling, access control).

4. Proposed Solution Design

- Deploy a cloud-hosted API layer in front of the inventory database; POS terminals and dashboards call this API rather than accessing storage directly.
- Use an API gateway as the single entry point, handling authentication (API keys/OAuth tokens), rate limiting, and request routing to backend microservices (inventory service, transaction service, analytics service).
- Store structured inventory and transaction records in a managed cloud database; archive raw logs and receipts in object storage for cost-efficient long-term retention.
- Feed transaction and stock-level data into a platform-provided analytics/forecasting service on a scheduled basis to generate demand predictions consumed by the dashboard.
- Secure store-to-cloud connectivity with site-to-site VPNs, restrict inbound traffic with firewall rules, and enforce role-based access so store staff and managers see only the data relevant to their role.

High-level architecture:



5. Conclusion

The proposed architecture separates client devices from data using an API-gateway-fronted set of services, stores structured and unstructured data appropriately, and uses a managed platform service for forecasting — meeting the case's requirements for efficient, secure, and scalable retail inventory management.

Answer 2: Cloud-Based Document Collaboration System

1. Understanding of the Case

- Multiple users must edit and share documents in real time from browsers across devices and locations.
- Documents need durable, versioned storage in the cloud with support for concurrent access.
- The system must remain fast and available globally, with strong security around who can view or edit each document.

2. Application of Cloud Concepts

- Cloud storage: Distributed cloud storage holds document content with version history, allowing rollback and audit of changes over time.
- Web APIs: APIs handle document CRUD operations, permission changes, and real-time synchronization events (e.g., via WebSockets or long-polling) between connected clients.
- Platform services: Managed identity and access management (IAM) services handle user authentication, while a content delivery network (CDN) and multi-region deployment support low-latency global access.
- Security: Encryption at rest and in transit, identity-based access control, and granular sharing permissions (view/comment/edit) protect document confidentiality and integrity.

3. Analysis and Trade-offs

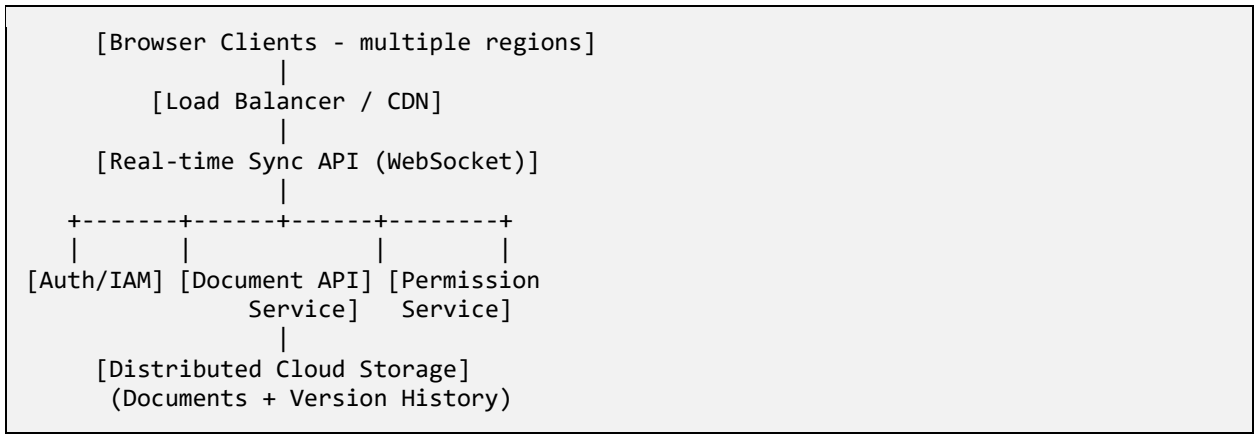
- Real-time collaborative editing requires a synchronization strategy (e.g., operational transformation or CRDTs) — this adds engineering complexity but is essential; without it, concurrent edits would conflict or overwrite each other.
- Storing full version history increases storage cost but provides recoverability and auditability, which is a reasonable trade-off for an enterprise collaboration tool.
- Multi-region deployment improves latency and availability for global users but increases operational complexity and cost versus a single-region deployment — justified here because the case explicitly requires global low latency and high availability.

4. Proposed Solution Design

- Build a web front end that connects to a real-time synchronization API (WebSocket-based) so edits from one user are pushed to all other active collaborators immediately.

- Persist document content and version snapshots in distributed cloud storage, with a versioning scheme that lets users view or restore earlier revisions.
- Use an IAM/identity service to authenticate users and issue access tokens; enforce per-document permissions (owner, editor, viewer) at the API layer.
- Deploy application servers across multiple regions behind a load balancer, and serve static assets via a CDN to minimize latency for globally distributed users.
- Encrypt documents at rest and enforce TLS for all data in transit; log access and permission changes for security auditing.

High-level architecture:



5. Conclusion

By combining a real-time sync API, versioned distributed storage, IAM-based access control, and multi-region delivery, the design satisfies the case's requirements for real-time collaboration, durability, global performance, and security.

Answer 3: Online Food Ordering Platform

1. Understanding of the Case

- Customers order food through web browsers or mobile apps; the platform must handle menu browsing, ordering, and payment.
- The system must securely store menu images, invoices, and customer data, and scale to handle variable order volumes.
- Backend services must be reliable and scalable, with strong authentication and role-based access for customers, restaurants, and delivery staff.

2. Application of Cloud Concepts

- Cloud storage: Object storage holds menu images and generated invoices; a managed database stores structured customer, order, and restaurant data.
- Web APIs: RESTful APIs handle menu retrieval, order placement, order status updates, and payment processing, consumed by both web and mobile clients.
- Platform services: Managed virtual machines and a managed Kubernetes service orchestrate backend microservices, enabling automatic scaling during peak ordering hours.
- Security: Authentication tokens (e.g., JWT/OAuth) verify user identity, HTTPS encrypts all client-server traffic, and role-based access control (RBAC) separates customer, restaurant-owner, and admin permissions.

3. Analysis and Trade-offs

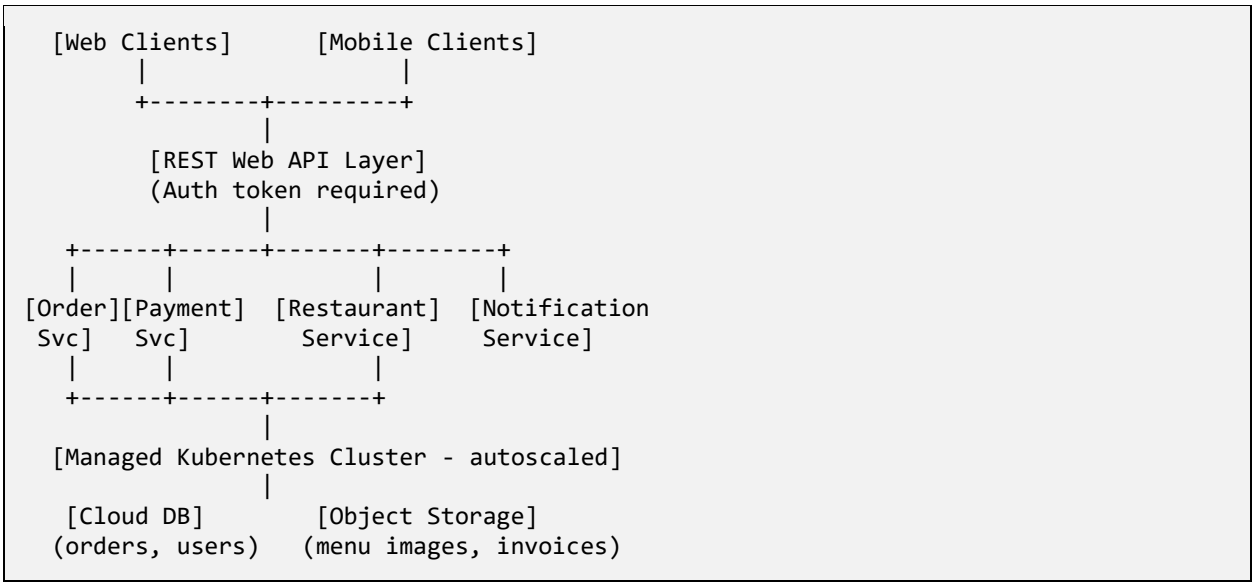
- Using managed Kubernetes for orchestration adds initial setup complexity compared to a simple VM deployment, but it pays off by enabling automatic scaling during demand spikes (e.g., meal times), which a fixed VM setup could not handle efficiently.

- Storing images and invoices in object storage (rather than the database) keeps the database lean and query-efficient, at the cost of needing a separate retrieval path for binary content — a standard and justified pattern for media-heavy applications.
- Token-based authentication (stateless) scales better across distributed microservices than server-side session storage, which is important given the system's multiple client types (web and mobile) and horizontally scaled backend.

4. Proposed Solution Design

- Expose RESTful Web APIs for menu browsing, cart/order management, and order tracking, consumed by both the web frontend and mobile apps.
- Deploy backend microservices (order service, payment service, restaurant service) on a managed Kubernetes cluster with autoscaling rules tied to request load.
- Store menu images and invoice PDFs in cloud object storage, and keep structured order/customer/restaurant records in a managed relational database.
- Issue authentication tokens at login and require a valid token on every API call; apply role-based access control so customers, restaurant owners, and admins each see only their permitted data and actions.
- Enforce HTTPS across all client-server communication and integrate a secure third-party payment gateway rather than storing raw card data.

High-level architecture:



5. Conclusion

The design meets the case's needs by combining scalable container orchestration, clear separation of structured and unstructured storage, and token-based, role-restricted security — supporting reliable operation from browsing through payment at variable order volumes.